

깊이 우선 탐색(DFS)

● DFS — 깊이 우선 탐색 (Depth-First Search)

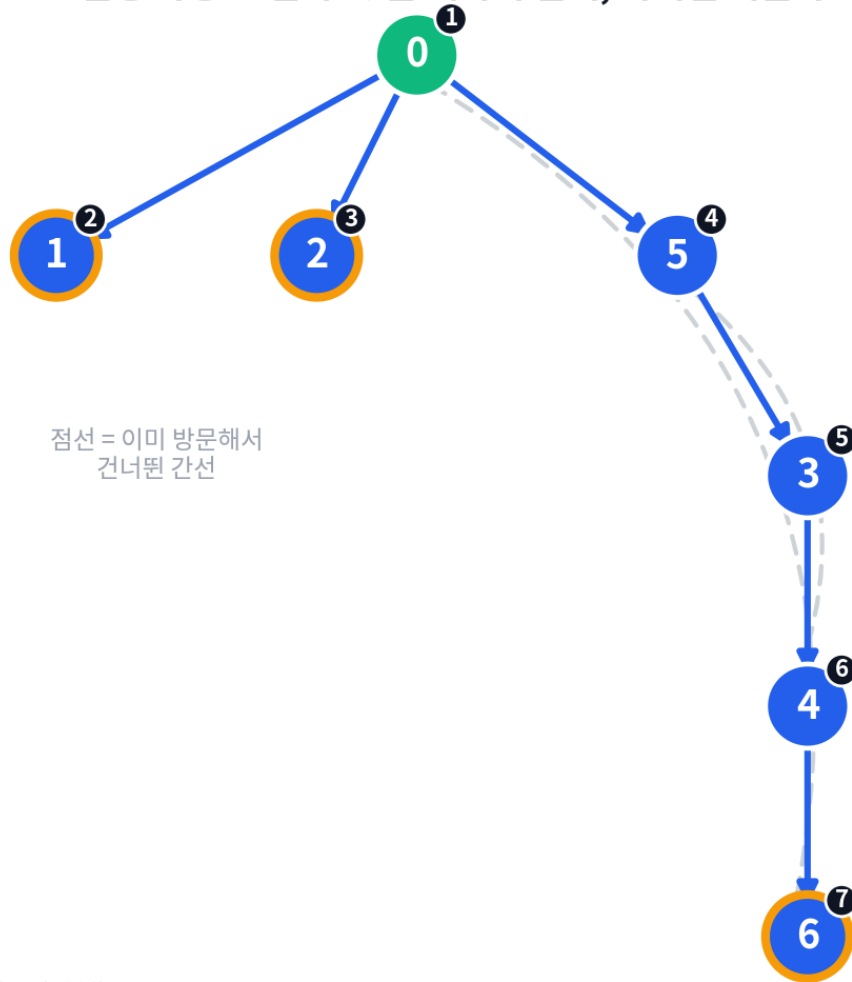
1. 핵심 아이디어

💡 **비유** — 미로에 들어갔다고 상상해보세요. 갈림길이 나오면 일단 **한쪽 방향을 정해서 끝까지** 가봅니다. 막다른 길에 부딪히면? 가장 최근에 지나쳐온 갈림길로 되돌아가서, 아직 안 가본 다른 방향으로 다시 출발합니다.

DFS는 이렇게 "일단 갈 수 있는 데까지 깊게 파고들고, 막히면 되돌아오는" 방식입니다. 넓게 퍼지는 BFS와 정반대로, **한 방향으로 최대한 깊이** 들어가는 게 특징이에요.

아래 그림은 위 코드의 `graph` (0~6번 정점)를 0번 정점에서 DFS로 탐색하는 과정이에요. 파란 화살표가 실제로 타고 내려간 경로, 회색 점선이 "이미 방문해서 건너뛴 간선"입니다. 1번과 2번, 6번(주황 테두리)에서 더 갈 곳이 없어 **막다른 길**을 만나고, 그때마다 가장 최근 갈림길로 돌아가는 걸 확인해보세요.

DFS 진행 과정 — 갈 수 있는 데까지 깊게, 막히면 되돌아오기



→ 먼저 방문 (진행)

○ 주황 테두리 = 막다른 길 (백트래킹 지점)

2. 어떻게 구현하나? — 재귀함수 또는 스택

DFS는 **재귀함수**나 **스택(Stack)**으로 구현합니다. 재귀함수 자체가 "함수 호출 → 함수 호출 → ... → 끝까지 가면 돌아옴"의 구조라서 DFS의 동작 방식과 정확히 맞아떨어져요. (이 노트에서는 재귀함수로 구현합니다.)

알고리즘 순서

```
DFS알고리즘 (현재정점 : v)
  점 v 방문 표시
  FOR t와 연결된 모든 정점 w
    IF w가 방문되어있지 않다면
      DFS알고리즘 (w)
```

Python 코드

```
def dfs(graph, v, visited, order):
    visited[v] = True
    order.append(v)
    for next_node in graph[v]:
        if not visited[next_node]:
            dfs(graph, next_node, visited, order) # 재귀 호출로 더 깊이 들어감

graph = [[1,2,5,6], [0], [0], [4,5], [3,5,6], [0,3,4], [0,4]]
visited = [False] * len(graph)
order = []
dfs(graph, 0, visited, order)
print(order)
```

⚠ 재귀함수를 쓸 때는 반드시 종료 조건(방문 체크)을 명확히 해야 합니다. 그렇지 않으면 같은 정점을 무한히 반복 호출하는 무한루프에 빠질 수 있어요.

3. BFS와 비교하면?

구분	BFS	DFS
탐색 방식	가까운 곳부터 넓게	한 방향으로 깊게
자료구조	큐 (Queue)	스택 (Stack) 또는 재귀함수
비유	물웅덩이의 물결	미로 탐험
잘 맞는 문제	최단 거리, 최단 경로	모든 경로 탐색, 조합/순열, 백트래킹
메모리 특징	한 번에 여러 정점을 큐에 쌓아둠	깊이만큼 함수 호출이 쌓임 (스택 깊이 주의)